

FOUNDER JOURNAL / 2024 TO PRESENT

A silhouette of a man in a suit, seen from the back and side, holding a glass of orange juice with a slice of orange and a cherry. The background is a light gradient.

The Build Before the Company

A record of the awakening, the failures, and the founding of Phielia

BY LUKE CLINE · FOUNDER & CEO, PHIELIA

"When something is important enough, you do it even if the odds are not in your favor."

QUOTE BY ELON MUSK, A PRINCIPLE I BUILD BY

Contents

I	The Awakening, My Story	11 parts
–	The Founder	Portrait
II	Insightful AI, the Company That Built the Founder	Stages 01–25
–	A Quote I Build By	Elon Musk
III	Palm AI, the Intelligence Layer for Global Real Estate Capital	2024
IV	Oracle Trader, Where Market Thinking Began	Q4 2024
V	MoodTrack+	End of 2024
–	Invictus	William Ernest Henley
VI	Almost the End of Vibe Coding	Jan–Apr 2025
VII	The Shift to Physical Products, Mason Lumière through Founder OS	2025
VIII	Cline Empire, Beyond Norme, and the Database Layer	2025–2026
IX	AGNTQS, the Real Operating Company	2026
X	The Public Record	2026
XI	Phielia and the Parallel Tracks	Present
XII	Additional Notes on the Build	Reference
XIII	What I Learned Along the Way	Closing

PART ONE

The Awakening

My Story

0 1

When Everything Stopped Making Sense

I was sitting in class, staring at the back of someone's head, when it hit me like a punch to the gut: why the hell am I here?

The teacher was droning on about something I knew wouldn't matter in five years, and I realized I'd been trusting this person with my entire future. But they gave up on life to work for a school. That's when the math stopped making sense.

I started thinking about people with Ferraris. How even if I became an open heart surgeon, the "dream job" everyone talks about, I'd have to save for a year just to afford what some people spend without blinking. I couldn't sleep at night. It wasn't just unfair, it felt like a lie delivered straight to my face, every single day, for years.

I was angry. I felt like I was being lied to by teachers who clearly gave up on life, telling me I needed good grades to succeed. It was ridiculous. The whole thing felt ridiculous.

0 2

I Started Digging, and the Evidence Was Real

I couldn't let it go. I started researching, and that's when I found it, a quote attributed to John D. Rockefeller:

"I don't want a nation of thinkers. I want a nation of workers."

Some people say there's no solid proof he actually said those exact words. But here's what I found that's documented as fact, at least as I understand it from my own research.

The General Education Board (1902)

- John D. Rockefeller created it with an initial one million dollar donation, worth thirty seven million today.
- By 1960, they had spent three hundred twenty four million dollars total, about seven point three billion in today's money.
- Frederick T. Gates, Rockefeller's advisor, wrote in the Board's 1906 official document: "In our dream we have limitless resources, and the people yield themselves with perfect docility to our molding hand."
- The Board's stated mission was the promotion of education in the United States without distinction of race, sex, or creed.

- What I came to believe the Board's actual mission was: to create a compliant workforce for industrialists.

What They Actually Did

- Funded over six hundred teacher colleges to train teachers in Prussian methods.
- Built over five thousand schools in the rural South, all with identical design: rows, bells, compliance.
- Took control of universities. The University of Chicago alone received eighty million dollars from Rockefeller.
- Controlled textbook publishers and curriculum selection.
- Replaced classical education, Latin, Greek, logic, rhetoric, moral philosophy, with standardized vocational training.

This wasn't just theory to me. This was documented institutional history, and once I found it, I couldn't unsee it.

03

First Principles Proof: Why This Is a Scam

First principle one: follow the money

When someone with nine hundred million dollars in 1902, Rockefeller was the wealthiest person in history at the time, suddenly invests hundreds of millions into controlling what children learn, you don't assume it's charity. You ask: what return does this investor expect?

The return, as I see it: a workforce that obeys authority without question, believes work provides meaning and purpose instead of ownership, accepts hierarchy as natural, and doesn't challenge the economic system that enriches the investor.

First principle two: incentives determine outcomes

The industrialists who funded education, Rockefeller, Carnegie, Ford, had one incentive: cheap, compliant labor. They didn't need philosophers. They needed workers who would show up on time, sit in rows, follow instructions, not question authority, and trade time for money.

First principle three: systems are designed for their intended outcome

The Prussian model, adopted by America in the 1800s, was explicitly designed to create obedient soldiers for war and compliant factory workers for industry. Not independent thinkers. Not owners. Not people who question the system.

First principle four: what gets measured gets optimized

American schools measure seat time, standardized test scores, and compliance. They don't measure financial literacy, critical thinking skills, wealth creation, ownership mindset, tax strategy, or asset building.

My conclusion at the time was that the system was optimized exactly for what it was designed to produce: workers, not owners.

04

I Saw the Full Cycle, and the Math Proved It

By then, I could see the script everyone was following, and the numbers didn't lie.

The script

- Go to school for twelve plus years.
- Get a college degree, four years, thirty to over a hundred thousand dollars in debt.
- Get a corporate job, or more school.
- Stay in debt for decades.
- Work until you die.
- Wonder why millionaires and billionaires exist while you're stuck.

The math

- Average U.S. household income: about seventy five thousand dollars a year.
- Average student debt in 2025: over thirty seven thousand dollars per borrower.
- Total U.S. student debt: one point seven to one point eight four trillion dollars.
- Forty three percent of college graduates are underemployed.
- Eighty five percent of workers are disengaged from their jobs, according to Gallup.

The ROI reality

One third of U.S. college programs show negative return on investment once you factor in opportunity costs, according to Brookings and the Webber Zou studies. That means for many degrees, you would be wealthier if you never went to college at all.

The math never added up, because as far as I could tell, it wasn't supposed to. The system wasn't broken. It was working exactly as designed.

Why Jobs Feel Like Scams, Proven by Economics

Here is what I figured out about being an employee, backed by how wealth actually works.

The employee trap

- You trade time for money, the one thing you can't get back.
- Your income is capped by salary bands, "that's the budget."
- You're taxed first, then you spend what's left.
- You build someone else's wealth, not your own.
- You're replaceable, there's always someone cheaper.

The wealthy playbook

- Own assets that generate income while you sleep.
- Use debt strategically, other people's money.
- Get taxed last, business expenses first.
- Build systems that work without you.
- Be irreplaceable, because you own the system.

First principles wealth math

Wealth equals assets times leverage times time.

Employees: wealth equals capped salary, minus taxes, minus expenses.

Owners: wealth equals assets times leverage, minus taxes, minus expenses.

Even a six figure salary can't compete with leveraged wealth. A surgeon making five hundred thousand dollars a year still trades hours for dollars. A business owner or investor can make millions without being present.

That's when I realized: jobs aren't broken. They're designed to keep you dependent.

How Do People Actually Afford Ferraris?

This haunted me. Here's the math.

- Average U.S. household income: about seventy five thousand dollars a year.

- Ferrari price: two hundred fifty thousand to over five hundred thousand dollars.
- At that income, a Ferrari is three to seven years of gross pay, before taxes.

But billionaires don't buy Ferraris with salaries. They use business profits, pre-tax money. They leverage assets, borrowing against holdings at low interest. They write off expenses, the car becomes a business asset. They generate passive income, money works for them.

First principles

Wealth inequality isn't about income. It's about ownership structure.

A worker trades time, gets paid, gets taxed, spends the remainder. An owner owns assets, those assets appreciate or generate income, they borrow against those assets tax free, and they spend the borrowed money.

The gap isn't about working harder. It's about understanding that wealth comes from ownership, not employment.

I couldn't afford half of what these people had, even if I saved every penny for a couple of years, even if I had a "good job." It made no sense, unless I realized the game isn't about salary. It's about leverage.

07

What I Realized Changed Everything

AGE 17

School was never about education, as I saw it

- Critical thinking was discouraged, classical education replaced with vocational training.
- Creativity was punished, standardized curriculum, "follow the rubric."
- Obedience was rewarded, the Prussian model: bells, rows, compliance.
- Financial literacy was absent, no classes on taxes, investing, or wealth.

The real curriculum nobody taught me

- Money is a tool, not a goal.
- Assets beat income every time.
- Ownership beats employment.
- Leverage beats labor.
- Tax strategy beats hard work alone.

I was seventeen at the time when I first saw people with Ferraris. It made no sense. Even if I saved every penny for a couple of years, I couldn't afford half of what these people had, even if I had a job. It made no sense.

I couldn't sleep at night, feeling this uncomfortable, knowing there were people out there living like that while I was being told to "get good grades." I wondered how they viewed us, average people. It was an uncomfortable feeling. It made me angry.

08

The Uncomfortable Truth, Backed by Data

It's not just me. This is what the data showed me.

- A generation of people can't think critically, solve problems creatively, or imagine a different future.
- They buckle at the slightest inconvenience. They're not emotionally resilient.
- Teen depression doubled from 2009 to 2019.
- Eighty five percent of workers are disengaged from their jobs, according to Gallup.

The industrial education experiment has been running for over a century, and to me, the results are in.

Knowing this is painful. It's an uncomfortable feeling, realizing you've been lied to, that the path you were promised leads to a cage, not freedom. But it's also the beginning of actual power. Once you see the system, you can stop playing by its rules.

09

The Moment Everything Changed

AGE 17

I was seventeen at the time. I'd seen people with Ferraris. It made no sense. Even if I saved every penny for a couple of years, I couldn't afford half of what these people had, even if I had a job. It made no sense.

I couldn't sleep at night, feeling this uncomfortable, knowing there were people out there and wondering how they viewed us, average people. It's an uncomfortable feeling. It made me angry. I felt like I was being lied to, straight to my face, in school, being told I need good grades from teachers that clearly gave up on life to work for a school. It felt ridiculous.

Once you see the system, you can't unsee it. And once you know it's a lie, you have two choices: play along, or build something real.

I chose to build.

The Day I Stopped Playing Their Game

AGE 17

I was seventeen years old. I had just proven, to myself, with evidence, with first principles, with math, that everything I'd been told was a lie.

School wasn't education. It was indoctrination.

Jobs weren't opportunity. They were servitude.

Debt wasn't investment. It was a trap.

And the people at the top? They weren't smarter or harder working. They just knew the rules of a game I was never taught.

I sat in that classroom one last time, looked around at the rows of heads bowed over textbooks, bells waiting to tell us when to move, when to speak, when to think, and I made a decision.

I would not be a worker.

I would not be a number.

I would not trade my life for a paycheck and call it success.

I closed my notebook. I stopped taking notes. I stopped chasing grades from teachers who had given up on life to work for a system designed to keep me poor.

That day, I died to the script they wrote for me.

And something else was born.

This Is Where My Entrepreneurship Journey Began

I stopped trying in school. Not because I was lazy. Because I finally understood the game. School wasn't teaching me how to win, it was teaching me how to serve someone else's dream.

The evidence, as I saw it, was clear:

- Industrialists funded education to create workers, not thinkers.
- The curriculum was designed for compliance, not critical thinking.
- The financial outcomes prove the system extracts wealth from students, rather than creating it.

- Wealth comes from ownership and leverage, not employment.

So I stopped chasing grades. I started chasing freedom.

This is where my timeline shifts. This is where I stopped being a student and started becoming an entrepreneur.

This is where the real story begins.



Luke Cline

founder & CEO

Luke Cline

FOUNDER & CEO, PHIELIA

AGE 19

I did not need permission to become what I wanted to become. When I reached something I did not know, I learned it. When I hit something I could not build, I learned how to build it.

PART TWO

Insightful AI The Company That Built the Founder

"This company didn't become my legacy. It prepared me to build one."
2024, 2025, Present

STAGE 01 TO 04

Ideation, Validation, Brand Foundation, Company Formation

I started by laying out the foundation, defining the core problem, shaping the long term vision, and identifying the initial target market. Around this stage, I mapped out the positioning for Insightful AI, an emotional intelligence platform. I called it the Google of emotions, setting the category with zero direct competition and creating a product institutions rely on to manage emotional climate risk. I called it the Tesla of therapy, establishing a premium brand halo. And I called it the OpenAI of healing, building the platform where therapists either license the technology or risk being left behind, driving deep user retention.

Next, I moved into customer discovery and demand testing. I focused on interviewing prospective customers, testing the problem and solution fit, and validating real signals of market demand before going further.

With the core validated, I established the brand identity. This meant choosing and clearing the official company name, securing the necessary domains and social handles, and designing the initial visual identity.

Finally, I handled the formal setup, registering the legal entity, drafting founding documents, and opening business banking accounts. As part of this milestone, I locked in insightful.com.co to secure the digital footprint.

STAGE 05

Product Architecture

From there, I mapped out the technical and structural blueprint. I defined our technical stack and infrastructure, designed the UX and UI system, and drafted the product architecture blueprint. Connected directly to this step was our core system engine, built under what I called the Insightful AI Pillar Protocol V999.9. A user's journal entry fed into a system I called the Truth Engine, which validated both emotion and factual accuracy. That data moved into what I called the Perfection Shield to filter insights, which then drove a Sentient UX Signal that adapted to the user's mood. This feedback loop was continuously influenced by a Reputation Engine that tracked a core trust score, ultimately leading to an Insight Response through a layer I called Godmode, designed for deep self awareness and real time self correction.

STAGE 06 TO 08

MVP Development, Initial Alpha, Website Launch

With the architecture locked down, I moved into physically building the first functional version, focusing on core functionality, assembling a working prototype, and preparing for internal testing. I established the initial Free Plan interface, which offered basic AI insights, basic mood analysis, and core journal storage. I grounded the product experience in a foundational philosophy prominently featured right on the interface: "The moments we take to reflect are the moments when we truly understand ourselves."

Once the MVP prototype was ready, I rolled out an internal alpha test to put the system through its paces. The focus was entirely on founder led, hands on testing, surfacing and logging critical bugs, testing edge cases, and validating our core user workflows in a live environment.

With the core experience stabilized, I shifted to public readiness by launching our primary marketing site and publishing key landing pages, and established our initial SEO and analytics foundation. The live site featured a clean, inviting landing page titled "Welcome To Insightful, Your AI Powered Journaling Companion," complete with a direct call to action, "Join The Beta." It displayed clear tiering cards, Basic, Pro, Freemium, and included an email sign up section, "Stay in the Loop," to capture lead interest and build an early waitlist.

STAGE 09

Public Identity

From there, I established our public presence across key channels, creating official social hubs on LinkedIn, X, Instagram, and YouTube, setting up a central content hub, and defining our core brand voice. Connected to this stage was a founder profile card, an editorial post titled "Luke Cline: The Mind Behind the Machine with a Soul, Insightful AI," anchoring the public brand to my personal story and putting a human face to the company as the brand voice went live.

A snapshot of our digital footprint at that time showed how the world was seeing the brand. Search results highlighted the story behind the project, profiling me as the eighteen year old founder and CEO behind Insightful AI, and showcasing our vision for what I called an Emotion OS. Direct links to my LinkedIn, Twitter, and Instagram accounts anchored the company's vision to a public persona.

This was the moment I shifted from building in private to leading in public. Up until this point, the platform existed behind closed doors, moving through ideation, validation, system design, and early alpha tests. Launching our public identity gave the company a human face. By verifying founder profiles and establishing a searchable digital footprint, I built the public trust, credibility, and top of funnel visibility I believed I needed right before inviting our first real users into the private beta.

STAGE 10 TO 11

Private Beta and Iteration

Key milestones here were to onboard invited early users, run structured feedback loops, and collect usage and behavior data. What I was advertising underneath this stage was a gradient landing card titled "Join the Insightful Beta," with messaging that read: "We're building Insightful, a smarter way to think, decide, and grow. Be the first to get early access, sneak peeks, and updates as we launch. Let's build something great together."

After opening the doors to our beta testers, this phase focused on rapid adaptation, taking the real world usage data and feedback from early members to squash bugs, refine UX details, and polish core functionality before scaling up further.

STAGE 12

Feature Expansion

With our foundation proven, I expanded Insightful AI into a multi layered ecosystem, scaling both the consumer app and our clinical business to business capabilities.

Subscription model and tiered access

I built out our commercial framework to support sustainable growth, structuring three clear tiers. The Free Plan covered basic journaling, basic mood tracking, basic AI insights, and entry storage. The Basic Plan, at four dollars and ninety nine cents a month, unlocked voice to text journaling, enhanced AI insights, basic mood analysis, and unlimited journal entries. The Premium Plan, at nine dollars and ninety nine cents a month, provided the full emotional intelligence suite, adding advanced AI insights, advanced mood analysis, AI generated artwork, voice to text, and unlimited entries.

The Pillar Protocol V999.9 system flow

I formalized the internal intelligence architecture that processed user input. Raw user input passed into the Truth Engine, which parsed both factual narrative and emotional weight. The Perfection Shield filtered the data to isolate actionable insights while discarding noise. The Sentient UX Signal and Reputation Engine dynamically adapted the app interface to the user's current mood state, continuously calibrated by a trust score. The Godmode Layer functioned as a self aware, self correcting feedback loop that generated the final Insight Response.

Interactive interface and voice suite

I implemented a workspace called Reflect and Grow, giving users an intuitive layout featuring "Express Your Thoughts," a voice recording and audio transcription UI with real time speech recognition, and a Live AI Insights Panel, real time feedback cards providing cognitive reframing prompts, for example, "What's one thought that's been particularly heavy?"

Therapist dashboard and business expansion

I expanded our scope from a consumer app into a clinical tool, introducing what I called Insightful WS Professional. This included real time client alerts, monitoring severe emotional signals such as flagging high severity crisis alerts, for example detecting passive suicidal ideation or hopelessness, so providers could acknowledge or mark a patient as resilient. I positioned the underlying framework as what I called the Neural Heart of Humanity, a living, self healing framework containing an underlying Planetary Emotion Grid, mapping real time emotional data on a global scale alongside specialized language engines and what I described as sentient AI mentors.

STAGE 13 TO 14

Go To Market and Content Engine

Next, I focused on building our initial launch engine. The key milestones included defining our core brand story and value proposition, finalizing our free versus paid tier structures, and designing a seamless onboarding experience to convert new leads into active users.

With the messaging established, I turned on our content flywheel to build consistent organic reach, publishing regular content across active channels, expanding our reach via targeted SEO and long form newsletters, and establishing an early, dedicated community presence.

Beneath this stage was a snapshot of our expanding social proof on Instagram. We scaled the platform to fourteen point seven thousand followers across five primary posts, gaining over one point nine thousand account views within a thirty day window. The bio read: "Insightful AI, Emotional Intelligence, Engineered. Mental Health x AI x Performance. Changing the world," alongside direct links to our website and an acquisition announcement with Kathleen Jones. I built out curated profile story highlights including The Journey, Ads, My Story, and Beta Access to turn casual profile visitors into early platform users.

STAGE 15

Revenue Engine

With momentum building, I turned my attention to designing our commercial model and preparing the business side of the platform, building a structured sales process to eventually convert users into paid subscribers, formalizing our marketing funnel, and putting together revenue forecasting to prepare for a seed round.

Our main Instagram scaled to sixteen point five thousand followers, establishing an active audience hub before the platform even went live. The hype hit a major turning point when our teaser content broke through, sparking a viral notification surge with over one thousand likes on a single reel.

STAGE 16 TO 18

Team Building, Operational Scale, Growth Stage

With early interest growing, I mapped out the framework for scaling the company beyond just myself, laying out the strategy to eventually bring on our first core employees, hire key department leads across engineering and marketing, and establish our foundational culture centered on high execution and bold innovation.

To make sure the platform could handle future traffic seamlessly, I spent this phase building out our underlying operational system, documenting our core processes, automating repeatable workflows, and building scalable infrastructure so the system would remain fast and reliable when public enrollment opened.

From there, I laid out the strategy for driving aggressive adoption and expanding our footprint, mapping out the key milestones to expand our customer base, form strategic partnerships, and grow our core revenue base to sustain the business long term.

STAGE 19

Expansion

Next, I set my sights on scaling beyond the initial consumer product by moving into broader markets and enterprise opportunities, establishing the vision to launch new product lines, enter new markets, and acquire enterprise customers, specifically positioning the platform for clinical and institutional adoption.

Tied to this milestone was the architecture map for our clinical business to business rollout. Client journaling and mood logs fed directly from the client account into the central Insightful AI engine. The AI engine processed this data to run real time emotion analysis and summaries. These insights populated the therapist insights dashboard via a secure therapist account, while all sensitive records were routed through a HIPAA compliant database. The system closed the loop by sending automated alerts and weekly reports directly into the therapist portal, giving practitioners actionable data on high priority client signals.

STAGE 20 TO 21

Maturity and Exit Preparation

From there, I focused on stabilizing the core operations and locking in steady, predictable growth, maintaining stable recurring revenue, operating mature internal systems, and sustaining a steady long term growth trajectory to ensure the business was built on a solid foundation.

Next, I moved into getting the business completely ready for an acquisition or liquidity event, completing a full financial cleanup and audit, ensuring legal and IP readiness, and engaging M&A advisors and business brokers.

STAGE 22

Broker Representation

Once exit preparation was wrapped up, I officially brought on professional advisory to run the sale process, executing a formal broker agreement, having the company independently valued, and finalizing all sales and marketing materials to present to prospective buyers.

Tied to this milestone was the executed legal contract initiating our sell side process, a formal Business Team Representation Agreement, an asset sale granting exclusive rights to broker the business. It identified the business as Insightful AI, based in Fullerton, California, listing me as the sole owner and authorized signatory. It formally set the listing price at four million dollars, granting the buyer full exclusive ownership of all intellectual property, source code, branding, and core assets upon closing.

STAGE 23

Recognized Leadership

In parallel, our market authority reached its peak, turning Insightful AI into a recognized, credible name across the space. I earned industry recognition, established myself as a thought leader in AI and emotional intelligence, and built a buyer facing market presence.

Search engines and AI summaries across the web recognized my role, highlighting early 2026 coverage of me leading Insightful AI, developing our proprietary Emotion OS technology, and driving our vision for AI driven mental health. High profile interest and buyer conversations surfaced directly, including direct messaging exchanges with prominent figures like Kathleen Jones discussing the valuation, the blueprint, and scheduling live deal calls to negotiate the acquisition.

STAGE 24

Acquisition Process

With serious buyers at the table, I moved directly into active deal execution, sitting down with prospective acquirers, negotiating core deal terms, and navigating the full due diligence process to ensure every piece of code, IP, and legal documentation was fully vetted.

Tied to this stage was a screenshot of live text messages with a prospective buyer, Kathleen Jones. In one exchange, she noted, "the medicine but you have the blueprint, it's valuable but only if followed what is told." She followed up to coordinate next steps with prospective acquirers: "As we get buyers interested, we can schedule a Zoom meeting so that you can explain this. They will have questions."

2026, Present

Key milestones at this point were being actively represented by a broker, positioning the company for acquisition, with a target range of four million to over ten million dollars. Bringing the journey up to the present moment, the company was actively represented on the market by our broker. I positioned Insightful AI as a turnkey asset, targeting an acquisition valuation range of four million to over ten million dollars.

Beneath this stage was our public listing profile on BizBen, listed as a pre revenue SaaS emotional wellness application originating out of Orange County, California, presenting the full Emotion OS platform to institutional buyers and prospective acquirers.

Reflecting on the Journey and What Lies Ahead

Looking back from where I stand today, the business has essentially been on hold with broker representation for about a year now. While it's certainly a shame that things haven't fully crossed the finish line yet, the experience itself was invaluable. I learned an immense amount through every stage of building, scaling, and negotiating this platform.

At any given moment, an offer could still come through and create the major breakthrough I've been working toward. But even if it ultimately never sells and nobody steps up to bring this vision out into the world to help people the way I originally intended, I'm completely at peace with that. I always knew that whatever happens, God has a reason for the timing and the path. Everything I built and every lesson I learned along the way were necessary steps to shape who I am and prepare me for whatever comes next.

“

*When something is important enough,
you do it even if the odds are not in
your favor.*

ELON MUSK, A PRINCIPLE I BUILD BY

Q2 2024, LATE 2024

Palm AI, the Intelligence Layer for Global Real Estate Capital

"Building a company taught me lessons no classroom, book, or course ever could."

Vision, Q2 2024, the idea

Right after I put Insightful AI on hold with my broker to wait for the right buyer, I didn't just sit on my hands. I immediately poured my energy into a new opportunity I spotted in luxury real estate across Palm Jumeirah. I envisioned applying artificial intelligence to that market by building a centralized system designed to streamline how high end brokerage agents manage their VIP clients, orchestrate multi million dollar listings, and automate complex daily operations.

Commitment, Q2 2024, going all in

While working at Eggs Up Grill, a small breakfast place near my home, I reinvested nearly every single paycheck back into the venture. I completely self funded development, paid out of pocket for all the AI tools and backend infrastructure, and committed fully to building Palm AI from scratch, doing it all without a traditional technical background or an external support system.

Execution, mid 2024, building the product

From there, I moved straight into execution. I built and iterated rapidly using Replit alongside AI assisted development tools, focusing entirely on turning the raw idea into a working product through constant experimentation and self teaching. I gave it everything I had, but during this phase, there was no team formed, no users onboarded, and no external interest generated yet.

Outcome, late 2024, venture concluded

Ultimately, the product didn't end up reaching users, attracting market traction, or validating demand. The venture concluded at the earliest stage of the startup lifecycle, before distribution, adoption, or team formation, revealing the real gap between building a product and earning true market validation. It was a tough outcome, but another essential lesson in my journey as a builder.

Compounding, 2024, foundation for what came next

"A failed startup isn't defined by what it becomes. It's defined by what it teaches you about what actually matters."

Even without users or initial traction, that entire experience became foundational. It taught me how to truly understand execution risk, market validation, and the hard realities of building a product without an established distribution strategy. It clarified what actually drives venture success far beyond just writing code, directly shaping how I approached future companies, like Insightful AI, with a much sharper focus on early validation, positioning, and real market demand.

Q4 2024

Oracle Trader, Where Market Thinking Began

Right after Palm AI concluded, I shifted my focus entirely to understanding financial markets and trade execution. That's when I started working on Oracle Trader. I wasn't trying to beat the market. I was trying to understand it faster than it could move.

The insight and shift

After seeing Palm AI fall short on distribution and market validation, I wanted to build something driven by real time data and pure system dynamics. Oracle Trader became the birthplace of my market thinking, a project where I began testing how AI could analyze signals, model behavior, and process complex market movements in real time.

Palm AI collapse context, and the shift

Right as Palm AI came to an end, I had to process an immediate emotional and operational reset. The project concluded without reaching traction, onboarding users, or building out a team, leaving me with zero market validation for that venture. Instead of letting that collapse slow me down, I used the immediate reset as fuel. It forced me to strip away all the fluff, face the hard reality of execution risk, and pivot instantly into my next focus with total clarity.

There was no pause period. Instead of taking time off to regroup or process the setback, I continued building Insightful AI alone while immediately shifting my cognitive focus into new technical experiments. I didn't stop or slow down. I kept moving straight into the next technical frontier, balancing my work on Insightful AI while actively diving into new trading and system architecture concepts.

Constraint reality, the single founder state

That brought me face to face with the hard operational reality of where I was standing. I was operating entirely in a single founder state, no team to delegate to, no funding buffer to cushion mistakes, and no structural stability. Every new concept, system, or iteration had to be architected and built completely in isolation, forcing me to rely solely on my own discipline, adaptability, and execution speed.

First market curiosity, expanding the scope

That's when I began exploring financial systems as a second domain outside of AI mental health systems. I wanted to test whether the underlying engineering, behavioral modeling, and AI architecture I had been honing could stretch into an entirely new arena, shifting from mental health framework design directly into the mechanics of financial markets.

Oracle idea formation, the core concept

That curiosity crystallized into a concrete vision, to build an AI system capable of interpreting market movements faster than human reasoning. It wasn't about simply chasing trends or making trades. It was about constructing an intelligent architecture that could synthesize complex market dynamics, detect real time signals, and process information at a speed and scale beyond traditional human analysis.

Core question shift, the new paradigm

That brought me to a fundamental shift in my thinking. The goal stopped being "can I trade" and became "can I simulate investment thinking." I moved away from standard execution and started building systems that could emulate how an experienced investor evaluates risk, analyzes strategy, and forms decisions in real time.

Buffett logic and long term value thinking

That evolution led directly into integrating Warren Buffett's core investment principles into the system architecture. Rather than focusing on short term market noise or rapid momentum, I introduced long term value thinking as a structural layer within the system, training the engine to evaluate fundamentals, intrinsic value, and sustainable moat logic above high frequency volatility.

Graham logic and risk management

From there, I introduced Benjamin Graham's classic investment principles into the framework. I integrated strict risk management and margin of safety rules to balance every decision. Before the system could ever entertain a speculative upside, it was forced to evaluate downside risk, calculate intrinsic value margins, and prioritize capital preservation above all else.

Momentum layer addition, a multi perspective system

To round out the strategy, I integrated short term price movement interpretation into the architecture. By combining Graham's risk parameters, Buffett's value logic, and short term momentum signals, I completed a multi perspective execution engine, one capable of synthesizing long term fundamentals with real time market dynamics simultaneously.

Technical architecture, the stack and flow

That multi perspective vision culminated in a clear, modular technical setup. I structured the system to handle ingestion, processing, and output in distinct layers: a data pipeline ingesting real time market feeds and financial metrics, a logic engine applying the fundamental risk filters alongside long term value scoring, and an execution and signal layer blending short term technical momentum with risk adjusted signals to output actionable intelligence.

Execution phase and tooling, the Replit build

With the framework designed, I immediately transitioned into building the actual prototype. I started building Oracle Trader in Replit using Python and AI APIs, leveraging rapid cloud development tools to connect real time data feeds directly into the multithreaded logic engine.

Data dependency and access reality

That's when I hit the first major architectural wall, data accessibility. I quickly realized that the system was constrained by public data feeds like yFinance level market data. Without institutional grade, low latency APIs or proprietary data pipelines, the engine's ability to execute true real time arbitrage or deep fundamental analysis was inherently capped by the quality of the incoming stream.

Stage Overview: MVP, March 2024, Single Script Intelligence

This stage marked the transition from theoretical framework design to building the first proof of concept prototype for Oracle Trader.

The objective and core hypothesis. The primary goal was to prove that a large language model could directly parse financial market data arrays and output coherent, actionable strategic reasoning. The scope was to strip away all system complexity to test the foundational link between raw market inputs and LLM evaluation.

Technical architecture and code flow. The setup was minimal, built as a single Python script running in Replit. The script prompted for a stock ticker symbol and pulled a seven day window of raw closing prices via yfinance, then injected that numerical array into a simple prompt template asking whether to buy, hold, or avoid, and explained simply, then passed the payload to OpenAI's API using gpt-4o-mini.

Key validations and working mechanics. The system successfully ingested stock data and rendered human readable strategic summaries without throwing formatting or interpretation errors. The LLM demonstrated an ability to recognize numerical trends, upward momentum, sharp drops, directly from a plain Python list.

Architectural limitations and gaps. The prototype lacked secondary guardrails, multi step agentic workflows, or mathematical processing before hitting the API. The Graham and Buffett rules designed in earlier phases were not yet embedded into the execution path. It was pure, unfiltered LLM output. And it relied entirely on public, rate limited, and delayed market data streams rather than institutional grade feeds.

Stage 00: Multi Thinker Prompt System

Once I had the raw script running, I knew simple data parsing wasn't enough. I needed to simulate how real investment minds think. I built a multi thinker prompt architecture designed to layer different strategic perspectives onto the data before generating a response, writing modular functions representing distinct investor personas for Buffett, Graham, and momentum analysis.

The goal was to create a single prompt structure that could simulate multiple investment heavyweights, Buffett for value, Graham for risk, and technical momentum for immediate trend direction. The win was successfully introducing conceptual layering into the system, forcing the model to evaluate the same asset through distinct strategic lenses. The gap was that the thinking structure was still disconnected from real time data logic, and the script lacked a synthesis layer to combine these separate perspectives into one final decision.

Stage 01: The Oracle Brain Concept

With the multi thinker framework tested, I realized running separate functions wasn't going to cut it. I needed to move away from isolated tools and combine everything into a unified reasoning engine, the Oracle Brain. I unified the logic so the AI would ingest the price feed and evaluate every perspective simultaneously within a single, coherent execution path.

The goal was to merge data fetching and multi perspective reasoning into one cohesive engine rather than juggling separate tools. The win was that the system officially became a real AI reasoning engine, taking raw market prices, running them through a multi perspective prompt, and returning a structured decision with supporting logic in a single call. The gap was that although the brain was unified, it was still running on an open loop, with no automated validation mechanism to test its past predictions against actual market outcomes.

Stage 02: Oracle Brain, Modularized

After unifying the brain, I realized having everything jammed into one block of code made it messy to scale. I needed a clean, modular structure where separate modules handled specific analytical duties before synthesis, breaking the code down into distinct imports and functions so each thinker lived in its own space.

The goal was to build a modular system architecture where data pipelines, individual thinker modules, and the synthesis layer were completely decoupled. The win was that the system gained clear structural design, letting me adjust how Buffett calculated value or how Graham managed risk without breaking the rest of the application. The gap was that it still lacked memory, scoring mechanics, or an automated improvement loop. It was a modular shell without historical context.

Stage 03: Pseudo Intelligence Engine

With a modular structure in place, I attempted to wrap the entire flow into a formal class based application, giving it the structure of a complete software product, encapsulating everything inside an OracleTrader class.

The goal was to make it feel like a real system, a self contained object oriented application that managed state, data fetching, and decision output cleanly. The win was that the codebase looked production ready, with clean object oriented patterns, neat execution methods, and structured output signals. The failure point was that it created an illusion of intelligence. Behind the clean class, it was still just a language model processing text prompts over market data with no real trading execution, backtesting edge, capital allocation logic, or user feedback loop.

Core Insight Formation

Building out the pseudo intelligence engine hit a wall of hard reality. Behind the clean class architecture and multi perspective prompts, the fundamental flaw became glaringly clear. Without a feedback loop, no validation users, and no real trading execution environment, the system was physically incapable of evolving. I was running a static simulation, feeding prices into a prompt and getting text back, without any mechanism to track whether the predictions were right, wrong, or improving over time.

I realized that intelligence systems don't fail because the prompts are weak. They fail when they lack four critical operational pillars: feedback loops, automated tracking to measure prediction accuracy against real world price action over time; distribution, real users interacting with the system to create actual usage metrics and market demand; execution environment, live integration with broker APIs or paper trading frameworks to turn reasoning into actionable execution; and focused build cycles, moving past broad, theoretical experimentation and locking into tight, single hypothesis engineering loops.

Final Lesson, the Takeaway

This realization reframed my entire perspective on building AI systems. I wasn't limited by how deeply I could analyze market principles or how creatively I could structure prompts. The wall I hit was structural.

"The constraint wasn't my thinking. It was the system I was trying to apply it in."

An AI model wrapped in code is just a wrapper until it operates inside a closed loop with real data feeds, live execution, and continuous feedback. This final lesson marked the end of building intelligence in isolation and set the foundation for how I approach system architecture moving forward.

The Breaking Point and Transitioning to the Next Attempt

That realization didn't just end a build phase. It hit on a personal level. Up until then, wrestling with AI architecture, multi thinker prompts, and system designs was an exciting puzzle. But constantly hitting hard structural walls without the right infrastructure turns intellectual curiosity into pure frustration.

When a project stops being a creative challenge and starts feeling like an uphill grind, the mental exhaustion sets in. The spark drops, the stress builds, and continuing down the same path becomes unsustainable. Recognizing that shift wasn't a failure. It was the necessary reality check needed before changing course entirely.

Driven by the need to break past those structural limits, and end the endless manual execution loops, I completely reset my approach for the next phase. I stopped trying to force a static language model prompt into acting like an institution grade trading room. Instead, the focus shifted to building a true event driven execution pipeline. I began setting up lightweight middleware to connect live market webhooks directly to automated execution handlers, and I stripped the scope down to a single, easily testable quantitative rule with clear start and stop parameters to safely measure real world performance.

END 2024

MoodTrack+

"I wasn't trying to beat the market. I was trying to understand it faster than it could move."

The shift, structuring emotional data

Instead of chasing market algorithms, the focus shifted toward taking subjective human feeling and turning it into an organized, functional system. The goal wasn't to step away out of defeat, but to direct that analytical energy into a domain that felt immediately meaningful and structured.

Phase 1, core emotion system, end 2024

I built simple, direct input paths to capture emotional states and contextual notes without adding friction to the routine. I introduced streak tracking, habit loop mechanics to maintain daily consistency over time. And I developed an initial insights layer, an analysis framework to surface early patterns between daily habits and emotional fluctuations.

Phase 2, retention and gamification layer

With the foundation established, the focus moved to deepening engagement and ensuring the habit stuck over time. I introduced daily goal structures and missions to encourage consistent interaction, integrated streak mechanics paired with timely positive affirmations to build momentum, and designed the interface and feedback loops so that tracking felt dynamic and personally rewarding rather than purely administrative.

Phase 3, expansion into a wellness ecosystem

Building on the gamified core, the scope expanded from a single tracking app into a comprehensive, interconnected product suite. I expanded the platform to include dedicated tools like Mind Spa, Vision Board, and Mood Coach, creating a ten app premium suite, each targeting a distinct pillar of self care, and linked the individual tools together so user data and insights flowed seamlessly across the entire suite.

Phase 4, intelligence layer

Once the wellness suite was established, the next evolution was shifting the platform from passive tracking to active, predictive intelligence. I integrated personalized AI coaching capabilities to offer real time reflection, contextual coping mechanisms, and actionable guidance based on log history. I developed what I called a Wellness Intelligence engine to process historical logs, surfacing deeper behavioral insights and correlations between lifestyle habits and mood shifts. And I moved beyond hindsight analysis to offer predictive indicators, helping users anticipate potential stress spikes or emotional dips before they occurred.

Phase 5, monetization layer

Once the product ecosystem was fully built and functioning as an intelligent wellness suite, the focus shifted to turning the platform into a viable commercial system. I structured multi tiered access models to monetize basic usage versus advanced personalized analytics, designed subtle, high converting upgrade prompts at key moments of value, and implemented framework level licensing and entitlement controls to manage user subscriptions cleanly across all ten apps in the suite.

Phase 6, stability and reality check

With monetization live across the suite, the final phase brought everything face to face with real world operations, system maintenance, and user retention dynamics. I focused on optimizing backend architecture to handle cross app user authentication, sync operations, and entitlement checks without friction, moved beyond pure acquisition metrics to study actual user lifetime value and identify where engagement dropped off in the funnel, and shifted focus from rapid expansion to maintaining operational balance, evaluating the true cost of supporting a broad, multi app ecosystem versus doubling down on the highest performing core modules.

The Unifying Narrative

Looking back at the whole arc, from pulling raw market price arrays in a single Python script to architecting a ten app intelligent wellness suite, the underlying driver was always the same: taking complex, unstructured real world inputs and building systems to process them into actionable intelligence.

The quant phase started with financial algorithms and stock data, pushing language models to execute multi thinker strategy frameworks until hitting the structural limits of static data feeds and open loop prompts. The emotional data phase redirected that architectural energy into human wellness, building an interconnected ecosystem of tracking, gamification, AI coaching, and predictive mood insights. And the product and commercial layer scaled the vision into a full ten app suite with unified cross app synchronization, multi tiered monetization, and real world subscription mechanics.

It highlights a complete journey from raw code experimentation to full stack product execution.

INVICTUS

BY WILLIAM ERNEST HENLEY

Out of the night that covers me,
Black as the pit from pole to pole,
I thank whatever gods may be
For my unconquerable soul.

In the fell clutch of circumstance
I have not winced nor cried aloud.
Under the bludgeonings of chance
My head is bloody, but unbowed.

Beyond this place of wrath and tears
Looms but the Horror of the shade,
And yet the menace of the years
Finds and shall find me unafraid.

It matters not how strait the gate,
How charged with punishments the scroll,
I am the master of my fate,
I am the captain of my soul.

JANUARY 2025

Almost the End of Vibe Coding, the Ignition

This marked the start of a completely new phase, moving into rapid, high speed prototyping driven by sheer momentum. Over four months, I shipped thirteen apps across two platforms to find out exactly where AI built software stops and real engineering has to start.

It all kicked off in January 2025 with three ideas built across three afternoons. The workflow was simple: type a paragraph, watch Replit turn it into a working app, ship it, and immediately move to the next one. It was fast, frictionless, and genuinely exhilarating.

The initial spark apps

- Homework Explainer for Students, an interactive tutor where you could snap a photo of a tricky problem and get a step by step walkthrough, hints first and answers second.
- Daily Mental Health Check In, a lightweight tracker asking five quick questions on mood, stress, and sleep, using AI to spot underlying patterns before you consciously noticed them.
- Budget Coach for Low Income Households, designed to turn raw income and bills into a realistic budget while automatically matching users to local assistance programs they might qualify for.

The budget coach was the first one that felt genuinely real, entering income and fixed expenses to get a custom life budget built alongside instant matches for local food assistance programs.

Chapter 02: February 2025, Momentum

By February 2025, the pace picked up significantly. I took the same fast platform, but shifted to tackling much harder, real world problems, caregiving, business compliance, trade skills, and crisis response. These were actual workflows with real edge cases, but building them still felt easy and fluid. That confidence was crucial for what lay ahead.

The four key projects

- Care Plan Helper for Family Caregivers, built to log medications and symptoms for loved ones while translating complex medical jargon back into plain English.
- Small Business Compliance Assistant, designed to ask a few quick questions about a shop, then produce the exact licenses, taxes, and deadlines actually required.
- AI Tutor for Trade Skills, providing bite sized electrician, plumbing, and HVAC lessons that dynamically adjusted to how fast you were actually learning.
- Crisis Resource Navigator, a high leverage tool where, after answering a few questions, it immediately found nearby resources, housing, food pantries, legal aid, and drafted intake calls or email forms for you.

The Crisis Resource Navigator was the project I was proudest of in this batch, just a few questions in, it had already located three nearby resources and drafted an intake email to the county. It genuinely felt like real leverage.

Chapter 03: March 2025, Escalation

New month, new platform, new stakes. Rent was already late, and the checking account read zero. Onchain infrastructure stopped sounding like a buzzword and started sounding like a plan. Base44 could generate a smart contract as fast as Replit generated a landing page, and I never once slowed down to ask if that was a good idea.

The three onchain projects

- Micro Grant DAO for Local Nonprofits, small community grants, voted on and released onchain, no board meeting, no six week wait for the money to move.
- Freelancer Escrow Contract, client funds locked on day one and released on delivery, built so gig workers stop getting stiffed after the last milestone.
- Transparent Mutual Aid Ledger, every donated dollar tracked from wallet to pantry shelf, public and queryable by anyone who wants to check.

It actually worked for the happy path: deliver, verify, release. The UI showed one thousand two hundred forty USDC locked in contract, with wireframes and dev builds delivered and released cleanly. But it was completely untested for real world friction. What happens on a disputed milestone? I never built that path. I never got around to what happens when someone disagrees.

Chapter 04: April 2025, Redline

This was the month the habit stopped being free. Code that moves other people's money doesn't forgive a guess the way a homework app does, and by now, I was building for donors and nonprofits, not just for myself. I shipped anyway. Shipping was the only plan I had left.

The three April projects

- Onchain Rent Relief Escrow, stake in, verify hardship, funds release same day, built for the exact wait I kept running into myself.
- Cross Org Aid Router, matching donated supplies to the nearest verified need across shelters, pantries, and clinics in real time.
- Wallet Recovery Assistant for Nonprofits, helping small organizations without a technical team back up multi signature keys before a lost laptop meant losing the whole fund.

The Aid Router went live with real donors inside a week. But right there in testing, I found a clear flaw: nothing stopped the same supply batch from being marked "routed" by two different pantries at once. I saw it, patched the surface symptom, and never went back to check what else that fix had touched. I just kept pushing forward.

The Ceiling: Where "Just Prompt It" Stopped Working

By the end of April 2025, the illusion of vibe coding completely shattered. The speed was incredible, but the reality hit hard: relying solely on prompts to generate full stack software reached its absolute technical wall.

Prompting works amazingly well for standalone tools, mockups, and happy path scripts. But as soon as you step into multi user state synchronization, edge case dispute logic, or real financial safety, simple text prompts start breaking down and producing race conditions. Quick prompt fixes only hide flaws under the rug. Hiding a bug with another prompt without understanding the underlying state machine leads to fragile systems that collapse under real world pressure. This moment redefined the entire approach for me. Generative tools are fantastic for rapid exploration, but true production readiness requires real software engineering, explicit error handling, rigorous testing, and deliberate architectural design.

Four Months Later: Reality Caught Up with the Hype

By the end of April, none of the thirteen things I'd shipped had made a dollar, and the checking account still read zero. Vibe coding hadn't failed because the ideas were bad. It failed because I could ship an idea faster than I could understand what I'd actually built.

The January and February apps still run fine, mostly because nobody's rent or survival depended on them being right. But the March and April onchain tools sat in a repository I was still afraid to point real money at, not because the core ideas were wrong, but because I was, the whole time, running one prompt ahead of my own understanding.

The lesson was never to stop building with AI. It was to stop mistaking a working demo for a finished product. Real engineering isn't just generating code that runs. It's understanding the architecture deeply enough to guarantee it won't break when real lives and real money are on the line.

The Shift to Physical Products

Phase 1 of 6, Product Attempt: Mason Lumière

This was the first real business swing, a skincare brand built solo as a Shopify store, starting with zero prior sales or marketing experience. I built out the full storefront from scratch, focusing heavily on product pages, branding, and visual design under the assumption that shipping a clean, professional site would automatically drive traffic. The outcome was zero sales. It delivered a clear, lasting lesson: having a polished store and a live website isn't the same as actually having customers.

Phase 2 of 6, Pivot to Service: Founder OS

After the skincare store hit a wall, the move was to pivot away from physical inventory entirely and sell expertise instead. This meant launching Founder OS, a mentorship and systems coaching program modeled directly after high ticket coaching playbooks from creators like Brez Scales and Iman Gadzhi.

Building the ecosystem and infrastructure

I built out a complete system architecture featuring what I called The Free Vault, a collection of systems, templates, and automation standard operating procedures, as a lead magnet to get founders into the pipeline. I brought two friends onboard to help execute outreach, design marketing assets, and structure the core delivery frameworks. And I implemented a full multi channel automated nurturing engine across TikTok, Instagram, YouTube, and paid ads, using hidden micro commitments, exclusive gated communities, email sequences, and SMS automation to drive leads into closing calls.

The execution and outcome

I executed aggressive outbound outreach, running the playbook step by step until landing three paying clients, generating approximately six hundred dollars in total revenue across those first deals. While modest compared to multi figure funnels, this was a massive psychological milestone. It proved that packaging systems into a service could generate immediate, non theoretical cash far faster than waiting on software adoption or e-commerce algorithms.

Phase 3 of 6, Inside Founder OS: ClickMint Case Study

Building on the momentum of Founder OS, the focus evolved from general coaching to helping clients build actual, functional offer mechanics. This transition led directly to the first real proof of concept for building customized agency assets for clients, ClickMint.

One of the early Founder OS clients was deeply passionate about hypercars and automotive culture, but lacked a structured business model or service to sell. Rather than giving purely theoretical advice, the approach shifted toward building the operational infrastructure directly for them. I built a specialized thumbnail agency brand from scratch named ClickMint, targeting automotive creators and YouTube channels, and designed and deployed a complete agency landing page using Base44, showcasing custom thumbnail design packages, clear value propositions, "Custom YouTube Thumbnails for Automotive Creators," and direct call to action flows, "Claim yours today."

ClickMint became the first concrete proof that the service model worked end to end, moving from asking what someone is passionate about to building the custom business infrastructure for them.

Phase 4 of 6, Inside Founder OS: Fishing Lure Case Study

With ClickMint proving that building client infrastructure worked, the goal was to test if the model was actually repeatable across completely different niches. Enter the next client, a fishing lure business. A second Founder OS client came in with a totally different passion, fishing lures and angling, but was hitting the exact same wall as the automotive creator. They had domain knowledge and enthusiasm, but zero operational setup, sales funnels, or structured offer to monetize it.

I applied the exact same build it for them framework used in the previous case study. Instead of giving them a list of homework tasks, I built out the entire business infrastructure directly, setting up a dedicated agency and product landing page, defining package options, and structuring their client acquisition pipeline from scratch. It landed another paying client, generating roughly five hundred dollars in revenue. This proved beyond a doubt that the Founder OS methodology wasn't just a one hit fluke with ClickMint. It was a scalable, repeatable playbook for turning any niche interest into a structured, revenue generating service.

Phase 5 of 6, Turning Point: MacBook Purchase

With the revenue generated from Founder OS, the momentum turned into a permanent upgrade in operational capability. The six hundred dollars earned from Founder OS was sitting as raw income, the absolute first money ever generated from any of these attempts. I converted that cash directly into a MacBook, the very first hardware tool actually owned, rather than borrowed or improvised. The infrastructure phase began. Everything built afterward runs directly on this machine, serving as the foundational engine for all future technical execution.

Phase 6 of 6, the Reality Check

Coaching, vibe coding, Shopify, AI tools, I tried all of it, and none of it held. I was working non stop, unable to switch it off, while getting bullied at school and having almost all failing grades senior year, with only one person, Peyton, still believing any of this would work.

Burnout hit harder than at any point before it, angrier, more exhausted, still no traction. But instead of walking away, the decision was to stop vibe coding entirely and study what was actually working for people who'd made it. That study became the model for the next build, no more prompting a project into existence, but actually learning the skills underneath it.

First Real Build: Cline Empire

Directly after the reality check and closing the chapter on past failures, the rule was set: no more vibe coding. It was time to study what was actually working for high performance communities and build a real application from the ground up, with the core vision of an insightful AI right on your shoulders, acting as a personal, strategic advisor integrated directly into the workflow.

Looking for proven community models, the structure behind high scale networks like Andrew Tate's War Room and The Real World served as the architectural blueprint. The goal was to build a full scale platform, Cline Empire, featuring an AI system designed to sit alongside the user like a dedicated strategist.

I built the first application that wasn't vibe coded. For the first time, this meant actually learning the hard skills underneath it: setting up GitHub properly, structuring a real codebase, managing databases with Supabase, deploying via Vercel, and architecting proper site mechanics. Every past failure, Mason Lumière, Founder OS, ClickMint, and the fishing lure agency, became the exact checklist of mistakes to avoid.

The grind was relentless, working all day, unable to turn it off, still carrying anger and looking for that major breakthrough. Ultimately, it became clear that while the technical execution was a massive step up, the model itself was too aggressive. If the vision of an intelligent, AI driven suite was actually going to scale, it needed to be built better, not just harder.

Beyond Norme, the Database Layer

Instead of scrapping the Cline Empire codebase and starting from scratch, the decision was made to reframe it. The foundation was already built. It just needed to be repurposed into a single, unified system: Beyond Norme.

Traditional business software was completely fragmented. Stripe handled payments, Skool managed the community, HubSpot tracked the sales pipeline, and Zapier tried to stitch them together with delayed, fragile webhooks that constantly broke. I reconfigured the entire system into a single, unified database schema. One user record instantly held everything: community activity, financial metrics like lifetime value and churn, and real time sales pipeline states, all natively in one place using the original architecture. The result eliminated data sync latency entirely. When a member took an action, the business telemetry updated instantly with zero middleware or reliance on external webhooks.

Beyond Norme, V1 Minimal Infrastructure

With the database refactored, the next move was stripping away every piece of bloat to ship Beyond Norme V1. The objective was simple: build the absolute simplest architecture that could support an actual production launch without relying on fragile third party stacks. I built out a streamlined, minimalist

interface focusing on three core views: a pricing tier at forty nine dollars a month, a clean, frictionless checkout entry point; magic link authentication, replacing bulky password forms with a low friction passwordless login flow; and a unified owner dashboard, a dark mode, single pane workspace displaying real time access tiers and direct links into the private community workspace.

This wasn't a mock up or a vibe coded prototype anymore. It was a production ready, fully owned stack, no broken webhooks, no middleware dependencies, just clean, native code built to scale.

Beyond Norme, the Routing and Access Architecture

Once the minimal UI was in place, the technical focus moved to constructing a rock solid, production grade routing architecture. The goal was simple: ensure zero unauthorized access and handle user flows seamlessly based on authentication and system status. Every incoming request hit the entry point, driving three primary functional branches. Standard public traffic routed directly into the Next.js App Router root, serving the landing page for unauthenticated visitors. A proxy middleware layer acted as the primary security checkpoint, inspecting authentication states and gating rules in real time. And once an authenticated request passed validation, it routed into the department portal, where a policy based access control layer checked fine grained permissions, with validated requests wrapped inside a unified, secure, persistent state.

This architectural setup was meant to eliminate data leaks and provide a bulletproof, edge level security model, proving that software built from real engineering principles easily outmatches quick, vibe coded scripts.

Beyond Norme, the AI Layer

With the database and routing infrastructure locked down, the final piece of the architecture was bringing the core vision to life: an insightful AI running directly inside the system. Most software drops an AI chatbot into a sidebar, it reads text, but it doesn't actually know the business. Beyond Norme's AI was designed to run as a background agent with native, structural access to the unified database. There's no parsing documents or retrieval tricks, it just sees what's happening in real time. When something breaks, a funnel drop or a bad metric, the AI evaluates it against real key performance indicators and rewrites the automation or pipeline itself, without waiting on a person to notice first.

Beyond Norme, the Growth Layer

Once the AI layer was integrated with the core database, the next challenge was tracking customer acquisition without relying on disconnected analytics tools. Marketing spend and actual business health usually live in separate tools, leaving founders to eyeball customer acquisition cost and lifetime value instead of tracking them in real time. I automated ad spend metrics from Meta and Google Analytics to

route straight into the same central dashboard alongside membership status and revenue data. Customer acquisition cost and lifetime value get recalculated continuously against real retention and fulfillment costs, making budget decisions instant and precise, with zero guessing.

Beyond Norme, the Launch Roadmap and Scale

With the system fully unified, the architecture built, and the analytics live, everything converged into a single, execution focused launch timeline mapped into clear, chronological phases leading directly to May 1. Each operational layer was built sequentially: a public conversion layer hardening the landing experience and deploying qualification checkout flows, activation and member depth layers wiring tier routing logic and building operator profile diagnostics, and a retention and launch gate deploying habit tracking engines and streak accountability signals, leading straight into full smoke tests and live production deployments. The entire journey, from early vibe coding failures and Shopify dead ends to high ticket coaching setups, culminated in a fully owned, production ready operating system built on real engineering principles.

Beyond Norme V3, Enterprise Systems and Premature Scale

With the launch architecture mapped out, the project entered its most ambitious phase, Beyond Norme V3, focused on architecting enterprise grade systems before actual market demand truly required them. The vision was to move beyond standard startup setups and build an infrastructure capable of handling high volume, multi tenant operations from day one. I designed high throughput data channels, automated compliance pipelines, and expanded the AI layer into an active sales copilot capable of assisting real time execution. Building for massive scale forced a complete mastery of enterprise architecture, proving that the shift from vibe coding to real software engineering was fully complete.

The Reality, Zero Revenue and Pure Infrastructure

The one point two eight million dollars in simulated volume, active dashboard metrics, and enterprise features weren't live commercial numbers. They were simulation metrics built into the infrastructure to stress test the environment. Despite building out a massive, enterprise grade architecture, complete with a Cold Call Copilot, policy based access control routing, database syncs, and custom dashboards, there was zero dollars in actual revenue coming through Beyond Norme.

Instead of treating the lack of cash as a failure, I reframed the entire project. The outcome wasn't a venture funded software company overnight. It was proof of concept. The entire stack served as concrete evidence that a single operator could build, deploy, and engineer complex, production ready software completely solo. The money wasn't in the platform yet, but the asset was built: total technical capability, real engineering discipline, and a fully owned codebase ready to deploy the second a real market opportunity opened up.

The True Ending, Pure Technical Growth

Stripping away the imaginary valuation and simulated revenue reveals the actual result of the entire journey: complete, end to end technical self reliance. There were no real customer payments, no venture capital backing, and no viral product launch. It was an exhaustive, solo engineering sprint built on top of sheer persistence.

What began as guessing with vibe code, failing with a Shopify store, and scraping together six hundred dollars from high ticket coaching turned into the ability to design full database schemas, build production routing, integrate AI models, and deploy complex web applications completely on my own. The ultimate asset wasn't a revenue stream. It was the transformation from someone relying on shortcuts to a true engineer capable of building anything from scratch.

Beyond Norme, the Core Modules

With the engineering foundation locked in, Beyond Norme transitioned into a modular suite built to handle real business operations cleanly from one place. A Hiring and Talent OS built a metrics driven intake funnel designed to screen candidates against actual real time business needs rather than static, generic job posts. A Financial Intelligence module integrated a built in ledger that tracks cash flow and recurring revenue side by side with member activity, enabling instant health checks instead of monthly spreadsheet audits. And a Membership Environment created a faster, stripped back alternative to platforms like Skool or Circle, removing likes and fluff in favor of accountability tools that directly tie client goals to mentor tracking.

Ultimately, Beyond Norme stepped back from being just an experimental software stack into serving as the overarching holding company layer for whatever project comes next.

AGNTQS, the Real Operating Company

With Beyond Norme serving strictly as the holding entity, the actual operational weight shifted into building a real business from the ground up: AGNTQS. Beyond Norme no longer functions as the consumer facing product. It holds the structure while AGNTQS operates underneath it as the live commercial engine.

For the first time, the endless solo grind came to an end. AGNTQS was built alongside a cofounder and a founding engineer, shifting the dynamic from isolated trial and error to real team execution. At nineteen years old, after six complete phases of failures, pivot attempts, and an intense period of burnout, the ground finally felt stable, backed by a real team, actual code, and a clear path forward, currently driving toward a seed round.

Beyond Norme, the Automated Admissions Flow

While AGNTQS scaled commercially, the technical blueprint behind Beyond Norme was engineered for total automation, specifically replacing manual onboarding with zero touch AI screening. A user submits an application form via the frontend, an endpoint fires immediately, saving the record as processing in Supabase, dispatching an instant confirmation email through Resend, and triggering a background evaluation job. The payload routes into a Groq AI inference pipeline, scoring candidates against a ten dimension rubric, evaluating parameters like operator mindset, execution track record, problem clarity, and risk profile. Standard backend logic validates the AI schema and applies hard confidence thresholds: approved candidates receive an outcome email with a secure token link for instant access; rejected or waitlisted candidates get automated feedback with a ninety day reapplication cooldown; and manual review triggers an instant Slack alert for team escalation if confidence is low. The stack is powered entirely by native infrastructure, Supabase, Groq, Resend, Slack, Stripe, and Gumroad.

The Founding Team

After months of grinding in isolation, coding late into the night, and carrying every single piece of technical and operational weight on my own back, the founding team for AGNTQS officially came together. I built Beyond Norme from scratch, pushed the raw software architecture as far as a solo operator possibly could, and led the strategic pivot to spin off AGNTQS, as CEO. Joe Hauser came on as co-founder and was the catalyst for the entire shift. He looked inside Beyond Norme, spotted the Cold Call Copilot, and saw what I hadn't, that it wasn't just a side feature, but the actual flagship product that justified building an entirely new company around it. Youness Oukhadjou came on as founding engineer, equity only, pre-profitability, taking complete ownership of the enterprise grade rebuild to convert the raw initial code into a scalable, production ready system.

For the first time on this entire timeline, the solo grind was dead. It was no longer just one guy trying to figure out everything by trial and error. It was three people with distinct roles, total alignment, and a real founding team ready to build toward a seed round.

The Realization That Reframed Everything

Up until May 2026, I was putting all my energy into building Beyond Norme. I had built it as this massive, AI native execution OS, and the Cold Call Copilot was sitting right inside it. In my head, that copilot was just a single feature, one cool piece in a much bigger tool stack.

Then my cofounder took a look at that specific tool, and the whole picture changed. We realized that in an agentic era, where autonomous AI agents do the real work instead of humans just typing prompts back and forth into a chat box, that copilot wasn't just a side feature. It was the entire core product. Beyond Norme officially stepped back into its true role as a holding company. We launched AGNTQS

at the end of May 2026 as the real operating engine, built directly around the autonomous agent itself, mapping out the incumbent competitor from day one, aiming to displace Salesforce by replacing legacy CRM workflows with fully autonomous execution agents.

And most importantly, this is where the solo sprint officially ended. After all those late nights coding alone, this was the exact moment I stopped carrying the entire engineering and operational weight by myself.

The Product Architecture

I mapped out the exact architecture for AGNTQS. Beyond Norme Inc. sits cleanly at the top as our parent holding company, its sole purpose holding the IP and underlying assets, keeping the corporate layer completely separate. AGNTQS itself is the actual operational marketplace, the core agent platform we're taking to market, built as an à la carte API marketplace so enterprises can deploy custom execution agents modularly, without needing months of onboarding.

The core platform has two products under the hood: AGNTQS OS, the agentic runtime layer that runs the agents and orchestrates work between them, and AGNTQS Grid, our sovereign data asset, per enterprise isolated storage that becomes the actual moat. Nobody can replicate a vault they don't own.

The four marketplace agents enterprises actually buy are AGNTQS Voice, live call intelligence, the one agent with real, running code, currently about thirty percent of the way to enterprise grade; AGNTQS Memory, our cross call context engine that lets a call today inform a follow up tomorrow; AGNTQS Sales, our revenue execution agent that turns a transcribed call directly into a CRM update or routing decision; and AGNTQS Trust, our compliance and consent agent, carrying the most legal weight, but currently sitting with the least finished code of the four.

The Branding

I broke down the branding itself, how the name AGNTQS was engineered to carry the entire pitch through six characters and six layers. A for the platform layer, the umbrella. G for the OS layer, the ground layer, the runtime engine. N for the data layer, the node and the vault, our sovereign data asset. T for the product layer, the tools themselves, Voice, Memory, Sales, Trust. Q for the intelligence layer, the compounding quality loop. And S for the enterprise layer, scale, zero migration, zero massive IT project, zero manual data entry.

Where Things Stand

Roughly three and a half months of active development, over one hundred total commits, three days for the admissions pipeline build, two weeks for the voice complete rewrite. Younes, our founding engineer, is rebuilding AGNTQS Voice from a browser mock into an enterprise grade mobile product,

implementing the full data vault schema with proper access policies, proxy architecture for every external API call, a mobile app with CallKit and enterprise device management, and a complete trust and compliance system underneath.

Here's the honest state of things: age nineteen, building this while still absorbing everything the last two years cost. A team of three, CEO, co-founder, and one founding engineer working for equity, not salary. One real module, Voice, at roughly thirty percent of enterprise grade, with five more modules existing as architecture, not code. One active conversation, the AT&T pilot, with ten specific, named blockers standing between here and a demo. Equity only, no revenue yet, no round closed yet. The ten million to billions ramp is a thesis, not a forecast with numbers behind it. Competition, Five9 and NICE, are very likely already inside the exact building this pilot is being pitched to.

Arq, Built for Our Team

Simultaneously, I built Arq to migrate off of Slack. Long before AGNTQS became the external enterprise marketplace, Arq was the internal working name for our core system, built strictly for our team to run our own workflows, pipelines, and operations. It started as our internal command center, giving the four of us a single, unified place to operate before we ever packaged it up for the public market or institutional investors.

Slack just stopped fitting how we actually wanted to operate. We didn't want another chat window or an endless stream of disconnected notifications. We needed a system built for structural execution. That frustration is why Arq was born in the first place, built as our own internal command center to run operations, engineering, and workflows exactly the way we wanted to work, long before it ever turned into AGNTQS. What Slack couldn't provide: an AI first workspace, a company OS, progress tracking, projects and tasks, files and docs, a knowledge hub, team operations, all built for our workflow.

The Competitive Landscape

As of June 2026, I mapped out the real competitive landscape. Direct competitors in call intelligence and enterprise included Gong, sitting at two hundred fifty to three hundred fifty dollars per user a month and roughly three hundred million dollars in annual recurring revenue, recording and transcribing sales calls but with no mobile native capture and no sovereign data layer; Chorus by ZoomInfo, with the same structural gap; Salesforce Agentforce, our biggest long term risk, building the agent flywheel directly inside Salesforce, but requiring a full Salesforce commitment, an IT procurement project rather than a drop in tool; and Momentum.io, focused on zero manual CRM entry but post call only, with no real time coaching during the call.

Adjacent players included 11x.ai running autonomous outbound AI sales development representatives, Artisan AI doing outbound email and LinkedIn automation, ServiceNow's Autonomous CRM already deployed at Bell Canada, and Cignara doing real time AI voice agents for customer support rather than sales reps.

The white space nobody else had built, as I saw it: mobile native, auto launch capture, a sovereign data layer owned by us rather than trapped in the enterprise's own CRM, cross enterprise compounding where one client's data sharpens agents for the whole network, and zero migration friction, a rep downloads the app and takes calls, no IT project required.

Is the Timing Right

Telecom sits at a forty eight percent AI agent adoption rate, nearly double the twenty six percent cross industry average. The AI sales agent market has split neatly into outbound automation and post call deal intelligence. Live call capture plus a sovereign enterprise data layer is a third category that doesn't have a clear winner yet. That's a real, current window, not a hypothetical one.

Honest State of the Product

Today, there is one wedge product, Voice, at roughly thirty percent of enterprise grade, ten unresolved blockers standing between here and a live demo, and zero signed pilots. A ten million dollar first year and a compounding path toward nine or ten figures is the kind of claim that needs an AT&T contract behind it before it's said to an investor rather than sketched on a whiteboard. The strongest version of this pitch right now isn't the trillion dollar total addressable market slide. It's the specific, narrow proof that the wedge works on one telecom, told honestly about exactly where the product is.

The Origin Line

From Mason Lumière to here, every single phase before AGNTQS existed just to teach one lesson the hard way. A skincare Shopify store that taught the difference between a store and actual customers. A mentorship program that turned into the first real dollars ever earned. Two client case studies that proved the method twice. A MacBook that turned income into infrastructure. And a burnout severe enough to nearly end the whole attempt while still failing senior year grades with one friend left believing it would work.

None of that was wasted. It's the exact reason AGNTQS is being built by someone who already knows what not to do.

The Public Record

Looking at the full Miro board on the screen, every single piece tells the story of how this identity was forged.

On one side sits the full structural lifecycle of the codebase: the Next.js App Router branches, the proxy middleware checkpoints, the policy based access control models, and the PortalShell. Above it sits the Cold Call Copilot dashboard UI, with live sentiment tracking, deal probabilities, and confidence meters, alongside the automated hiring and admissions flow modules. It is proof of a solo developer who stopped guessing and started engineering production grade software.

Center stage is a headshot labeled Luke Cline, Founder and CEO, anchored next to live Google search and AI overview results detailing Beyond Norme and Insightful AI, the Emotion OS, the Neural Heart of Humanity.

The search snippets and LinkedIn screenshots capture the scale of what I put out into the world, targeting a valuation trajectory of three hundred to five hundred million dollars within three years, building toward a nine billion dollar long term vision, operating out of Los Angeles. The bottom right window shows the active search: "Luke Cline continues to lead Insightful AI," documenting the shift from a kid dealing with high school setbacks and failed Shopify stores to an eighteen to nineteen year old full stack developer launching platforms after single all nighters.

At nineteen, the screen does not just show a portfolio. It shows someone who turned every failure, every burnt out night, and every zero dollar simulation into pure technical discipline, ready to step into a real seed round with a real team.

What a Direct Search Turns Up

When you run a direct search on my name, the internet pulls up a public footprint that mirrors every module on that board. The results highlight me as the Founder and CEO of Insightful AI, developing what I have publicly titled the Emotion OS, or the Neural Heart of Humanity. Google AI overviews summarize it as an AI powered emotional wellness tool with real time tone analysis, cognitive distortion detection, and voice journaling designed to serve as an AI mentor.

The public record frames me as the Founder and CEO of Beyond Norme, positioned as an AI native execution operating system built to compete with and replace legacy community and course platforms like Kajabi, Circle, and Skool. The search snippets capture the narrative that I programmed the entire infrastructure independently, and that we launched the first public build in May 2026 after pulling a single all nighter.

My profiles, pitch decks, and posts explicitly publish a nine billion dollar vision, showcasing financial projections, automated hiring workflows, and a hyper scaled growth model. Search results index me out of Los Angeles as a young, independent founder. When someone Googles my name, they see a nineteen year old solo engineer who built an entire dual platform ecosystem from scratch, claimed full technical autonomy, and publicly put a multi billion dollar vision out into the world.

The vision behind Insightful AI and the Emotion OS

Across the board and my live pitch materials, I mapped out Insightful AI as the Neural Heart of Humanity. It was not just a wrapper. I engineered voice journaling, cognitive distortion identification, and real time tone feedback into a proprietary framework designed to scale toward a multi billion dollar market vision.

The infrastructure of Beyond Norme

I laid out the entire execution ecosystem built to replace tools like Skool, Kajabi, and Circle. The board details the proxy middleware, the policy based access controls, the Next.js routing, and the Cold Call Copilot dashboard UI, the exact system I programmed solo after pulling an all nighter to push it live.

The Compounding Intelligence Thesis

This is the foundational engine behind everything else I have built toward: agents go out into the enterprise, learn from real world operations, report back for human expert review, and get retrained and redeployed sharper than before. Every single cycle compounds, turning raw enterprise calls into an autonomous, self improving asset that legacy competitors cannot touch.

1. Deploy: specialized agents go into a client's real operations, calls, research, negotiation, workflows.
2. Learn: agents absorb real behaviors, decisions, edge cases, and outcomes from the work itself.
3. Return: what they learned comes back to a private, encrypted data layer, isolated per client.
4. Human review: people validate, redact, and prioritize what actually gets kept before anything trains on it.
5. Compound and train: only the validated signal updates the models, knowledge graphs and behavior get sharper.
6. Redeploy: sharper agents go back out. The cycle repeats, and the advantage compounds each time.

That is the ultimate vision mapped out on the board: building a system where every conversation makes the entire intelligence network smarter.

The Moonshot, and Where Things Actually Stand

The revenue ramp target was to hit an initial per seat pilot tier, scaling across five to ten thousand enterprise seats within AT&T, and capturing compounding overages from cognitive blocks. The valuation milestone was converting that pilot proof into a Series A backed by multiple enterprise logos and real, recurring annual recurring revenue. The network effect goal was reaching a point where the compounding feedback loop across deployments made the real time intelligence layer impossible for legacy desktop competitors to replicate.

That is the moonshot thesis, the version of the story that gets told once there is proof. As I have said elsewhere in this document, and say again here because it matters: today there is one wedge product, Voice, at roughly thirty percent of enterprise grade, ten unresolved blockers standing between here and a live demo, and zero signed pilots. A ten million dollar first year, and a compounding path toward nine or ten figures, is the kind of claim that needs an AT&T contract behind it before it is said to an investor rather than sketched on a whiteboard. The strongest version of this pitch right now is not the trillion dollar total addressable market slide. It is the specific, narrow proof that the wedge works on one telecom carrier, told honestly about exactly where the product is.

The Twelve Month and Three Year Vision

The twelve month vision: pilot converted to a full phase one contract, three to five additional enterprise customers across two to three verticals, two to five million dollars in annual recurring revenue, a Series A closed between fifteen and twenty five million dollars, a company valued between seventy five and one hundred twenty five million dollars, a second founding engineer onboarded, and a team size of eight to twelve people total.

The three year vision, looking toward 2029: fifty to one hundred enterprise customers across five or more verticals, fifty to one hundred million dollars in annual recurring revenue, market leadership in enterprise AI agent infrastructure, and an IPO or strategic acquisition under active consideration.

Hiring Economics and Legal Simplicity

My working principle on hiring has been that every hire must identify the bottleneck removed, the speed gained, the milestone unlocked, and the revenue or capital opportunity created. At current core capacity, engineering payroll runs lean, and it only scales up when a real constraint justifies it, not because there happens to be money in the account.

On the legal side, the objective is institutional venture capital, so the corporate structure should stay investor compatible and simple. A conventional Delaware C-corp is the standard venture backed structure. I have tried to avoid building a complicated multi entity international structure merely for appearance, and instead protect IP, founder ownership, equity documentation, employment and

contractor agreements, tax compliance, and investor rights. Complexity should only be added when a real business or legal requirement justifies it, since every additional system creates maintenance, failure modes, legal cost, accounting burden, and cognitive overhead that has to earn its existence.



Phielia

Phielia

By mid to late July 2026, moving into August, AGNTQS shifted slightly, and we changed the name to Phielia. The Phielia marketing and pitch page can be found at phielia.com, and Youness, my founding engineer, is finalizing the MVP. We currently have a way to take leads on a lead form, and we are now looking for funding, asking for one point five million dollars.

Phielia is a real time coaching layer for live enterprise conversations, starting with telecom sales calls. It understands what is happening as the conversation unfolds and gives the representative actionable guidance before the moment is gone. It is not another AI feature added to yesterday's software, not post call analytics like Gong, Chorus, or Momentum, not an IT first agent platform like Salesforce Agentforce or ServiceNow, and not an AI sales development representative like 11x or Artisan. Others analyze, automate, or replace. Phielia augments, in real time, at the point of conversation.

The technology, market, and architecture have converged. Real time in call coaching only became viable once speech to text latency fell into the low hundreds of milliseconds, a threshold that has now been crossed. Salesforce and ServiceNow are validating enterprise demand for agentic AI in customer workflows, but incumbent deployments still involve procurement, integration, and implementation measured in months. Phielia starts at the rep level, where value can be delivered immediately. And most conversation intelligence was built for post call analysis. Phielia is being built around a real time runtime and persistent context layer from the beginning, avoiding the architectural debt of retrofitting real time automation later.

The MVP is still being finalized. The opportunity is to prove one workflow, live call coaching, while the market is opening and before incumbent architecture becomes the default. Phielia is currently finalizing its MVP around one focused workflow: helping enterprise representatives perform better during live customer conversations. The underlying architecture is designed to support memory, execution, and compliance as independently validated capabilities over time, allowing Phielia to expand from a focused wedge without rebuilding the core. Prove the live conversation first. Expand only where the data says to.

The Founder and IP Control Package for Phielia has been refined, and is currently not signed and not yet used, but will be soon. The funding and execution planning documents that were originally written for AGNTQS still apply, just renamed for Phielia. We are still in process. We have not spent too much time yet looking into getting funding, and we know it is important. We are focused first on legal structure, and then finalizing the MVP is what Youness is doing.

 Phielia

Intelligence
that helps people
act before the
moment is gone.

LIVE

Real-time coaching layer
for live enterprise conversations,
starting with telecom.

PHIELIA.COM

Youness's Update, August 25, 2026

MVP, completed or substantially completed: the backend platform foundation is established, with logging, request IDs, and structured error responses implemented, and core security foundations in place. The authentication system is completed, and the approach was selected and mapped out. The authorization and security architecture has been decided and planned, with auth modules and security boundaries designed. The core backend architecture is established, with platform level modules being structured around auth, users, and organizations. And legacy cleanup is roughly ninety percent complete.

MVP, currently in progress: the migration from local storage to PostgreSQL is underway and expected to take several days. The secure recording and private storage pipeline has its architecture mapped out, with Cloudflare R2 integration being implemented for a private recordings bucket, and secure access credentials being set up, expected to be completed next. Once the storage pipeline is done, the full backend will be tested against the frontend, and integration bugs will be identified and fixed.

MVP, next features: a Notion integration is the planned next major feature, alongside background jobs. Final legacy cleanup will remove remaining unused components after feature work. AI voice diarization will add automatic speaker recognition to eliminate manual speaker switching. A capture solution decision still needs to be made, evaluating a browser extension, a desktop application, or an alternative capture architecture, followed by building whichever approach is selected. And CRM sync is the final major MVP feature in the current roadmap.

Overall position: completed work spans the backend foundation, security foundation, authentication, architecture, and about ninety percent of legacy cleanup. Current work is the PostgreSQL migration and the secure recording and storage pipeline. Next is backend and frontend integration testing, then Notion integration and background jobs, then cleanup, AI diarization, the capture solution, and CRM sync. We are past the foundational backend stage and into the actual product feature and integration stage. The immediate technical milestone is getting secure recordings into private storage and verified end to end through the frontend.

The Parallel Tracks

While AGNTQS, now Phielia, is the main engine, there is an entire second set of concepts developing right alongside it, starting with Orbitry.

Orbitry, the global scheduling engine

Orbitry is built for a completely different scale. Orbital factories exist now, but nothing is scheduling them. It is an optimization engine designed to decide what gets made, where, and when, across every factory in orbit and every launch window on the ground. The solo build plan is eight weeks, solo, on a MacBook: model the problem, build the solver core, prove it is better than nothing, make it visual, and

go public to start raising. The digital footprint strategy follows a strict order of effort and timing, first a working website, then LinkedIn to build the real ops presence, and finally X to post the solver hitting its first valid schedule and benchmark charts.

Astoria, the intelligence operating system

Astoria is not just a pivot or a side project. It is the overarching architecture that everything else could plug into, an intelligence operating system designed to coordinate specialized AI systems, memory, reasoning, and human interfaces into a single, continuously evolving platform. It rests on four pillars: goal driven, taking complex objectives and breaking them into coordinated execution; memory first, accumulating institutional knowledge across every execution and reflection; truth governed, all outputs passing evidence validation before being accepted or shared; and self improving, reflecting after every execution cycle and updating its own knowledge base. The control principle behind it is that it should remain observable and human governed. Every decision, reasoning step, and memory change should be traceable and auditable, with humans remaining strictly authoritative, because the system assists, informs, and recommends, but never decides unilaterally.

Ultimately, these were just ideas in my head. I never tried them, because I need to focus on what is real now, after so much effort. I am finally here, almost at the finish line.

Additional Notes on the Build

The Founding Four and the Target Organization

Despite how massive the full target organizational chart looks on paper, the reality of who was actually in the building at this stage came down to four people. Luke Cline, CEO, steering the overall vision, managing product direction, and leading the transition from Beyond Norme into AGNTQS. Joe Hauser, co-founder, operating alongside me, refining commercial strategy, and helping drive our positioning toward the seed round. Youness Oukhadjou, founding engineer, taking ownership of the frontend and full stack side as lead product engineer. And Brandon Nguyen, founding engineer, serving as lead systems and AI engineer, handling the deep backend architecture, server infrastructure, and core AI systems.

The rest of the target organization, mapped for where seed capital would go: a head of enterprise sales to build and lead the revenue engine, a VP of business development and partnerships manager steering strategic alliances, sales pods of sales development representatives, account executives, and sales operations, a product and UX team reporting under Youness, and a backend and AI systems team reporting under Brandon.

Seven Phases, One Codebase

Phase one was the unglamorous plumbing, the core foundation everything else depends on, done first and done on purpose. We initialized Supabase, scaffolded Next.js with TypeScript, laid down our first database migrations, stood up the CI/CD pipeline with Husky git hooks, automated validation, Sentry error tracking, and feature flags, and stabilized cron routes and cookie authentication.

Phase two was the nine department portal and the first Copilot, the first time the product actually looked and felt like a complete, working system. We shipped nine distinct operational dashboards, Operations, Growth, Finance, Creative, Mentorship, Technology, Sales, Contractor, and more, baked in role based and policy based access control from day one, and deployed the very first functional version of the Cold Call Copilot in browser.

The mid to late build was the complete rebuild of the live call copilot into AGNTQS Voice, migrating to Next.js with secure session auth, integrating Groq AI for orchestration, and building out knowledge base intelligence and transcript analysis, with the core system built around a large main orchestrator supported by many sub-components, custom hooks, and internal libraries.

The most recent shipped work was the cinematic portal, a public marketing surface on beyondnorme.com running a minimal, black and white design that revealed nothing about what was actually being engineered underneath, with a cinematic login gate and a strict two step auth model, a cover gate first, then the real portal vault behind it, with zero visible connection on the public page linking the cover site to the actual product infrastructure.

Current State: Seeding and MVP Finalization

As of that stage, the nine department portal system sat at eighty percent completion, the admissions pipeline at eighty percent, the cover and portal auth infrastructure at seventy percent, the database architecture at sixty percent, and AGNTQS Voice at thirty percent, the piece with the most engineering headroom left to clear.

The control plane, sixty migrations deep, was completely locked in, with enterprises, agent registry, deployments, billing, fine tuning jobs, global patterns, and internal users fully migrated and running in Supabase. The grid vault schema needed for real pilots, call sessions, call intelligence, customer profiles, deals, reps, pre call briefs, compliance log, temp audio buffer, CRM webhook log, and session events, was mapped out but still waiting on code.

The Ten Technical Blockers

The mobile app had zero code yet, an iPhone app with CallKit integration being the single biggest blocker to any live demo. The audio capture pipeline was running on browser MediaRecorder only, with no Twilio Voice SDK, WebRTC, or SIPREC. Real automatic speech recognition still relied on built in browser speech recognition instead of a real streaming Deepgram integration with speaker diarization. Structured extraction was running on Groq for text processing, but needed a real Claude API pipeline for fear, desire, and objection tagging. There were no dedicated server side proxy routes for external APIs, meaning API keys would be exposed client side. Enterprise data isolation policies were user level instead of enterprise level, a major risk for multi client deployment. The consent flow was built but not integrated into the transcription activation path. There was no PII scrubbing integration yet, required for compliance. The coaching overlay was currently just a mock UI. And there was no integration yet with enterprise CRM systems or telecom specific platforms.

The Roadmap, as of That Stage

Weeks one through four focused on finishing remaining consent wiring tasks, applying pending database migrations to production, deploying the full data vault schema with proper row level security, rewriting proxy architecture for every external API, and beginning the mobile app scaffold. Weeks five through eight focused on the Expo mobile app scaffold with native CallKit integration, Deepgram streaming transcription, a Claude API proxy with structured extraction, and PII scrubbing middleware. Weeks nine through twelve focused on a production ready coaching overlay UI, a full CRM webhook system, enterprise provisioning APIs, and comprehensive end to end pipeline testing. The ninety day

pilot target was a functional iPhone app under enterprise device management, real time call intelligence running on live calls, a compliance dashboard that passes legal review, automated CRM data pushing, and a control plane managing the pilot cluster, to secure a first signed, revenue generating pilot.

The intended enterprise expansion sequence was to prove the model first in telecom with an AT&T pilot targeting five to ten thousand seats, then expand vertically into logistics dispatch, pharmacy services, and wealth management, and finally scale horizontally toward government and defense, healthcare, and standard SaaS companies. The stated reasoning for telecom first: regulatory compliance requirements create a moat competitors cannot replicate quickly, telecoms require strict data isolation that only an enterprise cluster model satisfies, every sales or support call is a transaction with ROI measurable directly in close rates, and five to ten thousand seat pilots justify meaningful annual recurring revenue on their own.

The Funding Path, as Mapped

The current stage was seed, pre MVP, the founding team operating on equity only, taking zero cash salary while building toward the enterprise pilot. A signed pilot was intended to serve as the anchor proof point for the next raise. Post pilot conversion targeted a Series A raise, backed by a signed pilot, additional telecom logos, and early annual recurring revenue. Post Series A expansion aimed to scale across enterprises and verticals, setting the stage for a Series B with a clear line of sight toward an IPO or strategic acquisition later in the decade.

The architecture was also built with a divestiture ready core from day one, every enterprise record carrying a built in divestiture flag and automated data return instruction, so that if an acquisition, spinoff, or exit ever demanded it, the system could export every byte of that enterprise's data, generate a certified destruction certificate, cleanly tear down the cluster, and wipe the isolation policies, with the control plane transferring cleanly to an acquirer while enterprise data stayed with the enterprise or was securely destroyed exactly as contracted.

The long term horizon was never just telecom. It was every industry that runs on a phone call, telecom and infrastructure first, then financial services, logistics and supply chain, healthcare and pharma, and technology and enterprise cloud.

One nuance worth naming honestly: Salesforce shows up twice in this map, once as the incumbent we are positioning to displace, and once as a CRM integration partner we push structured call data back into. Both are true at different points in the lifecycle, and owning that tension clearly, rather than hiding from it, was part of the pitch discipline from the start.

CLOSING

What I Learned Along the Way

In my own words

On Permission

I realized I did not need permission to become what I wanted to become.

I had no degree. No institution had certified me as an engineer. So I built without one.

When I reached something I did not know, I learned it. When I hit something I could not build, I learned how to build it. I taught myself engineering, product, infrastructure, AI, and the rest of the stack by refusing to let a missing credential become a limitation.

The approach was simple: I stopped treating the rules around who is "qualified" as laws of nature.

I did not need someone to tell me I was ready. I could test that myself by building something real.

That decision changed the trajectory of my life.

For two years I have entered the unknown, having begun various things that eventually failed, lost friends, pushed people away, questioned myself, and spent thousands of hours trying to work out what was wrong with what I was doing. For the last seven to eight months I have been averaging about a hundred forty hours a week, teaching myself whatever came my way and forcing myself to improve at it. I did not earn my first dollar until about a year later, when I set up my own online coaching agency. There were nights when I was not sure at all if any of this would ever succeed. I still do not know. That used to worry me. Nowadays I believe that is exactly what it is all about.

I discovered that it is unnecessary to know where the road ends if I simply want to keep going on. Whenever there was no path, I created one for myself. Each time I failed, I analyzed the reason for the failure. Whenever someone told me no, I tried once more. When I realized that I could not build everything by myself, I learned how to find people who could build alongside me.

I have come to have a deep affection for the act of trying itself, since each failure has made me more capable than the person who began the journey. I am now actually building something that seems real together with my founding engineer, and I am aware of just how much it has cost me to reach this point. I would go through it all again, not because I am certain that I will win, but because I have at last found something worth fighting for.

On What I'm Building

When you put your blood, sweat, and tears into creating something, you're building something. It's like a child. And so it's like, which one am I going to let starve to death? I couldn't bring myself to do it. So, I split the money between the two.

I believe in it. I've seen how far I have come. I have learned a lot. I have a story now, and it's real. It hurts, what I do is hard, and it hurts a lot. I want this preserved, because it is meaningful to me. I want it worded so it means something, because it means a lot to me as a founder who has been through it, who has learned through trial and error, and who is now crawling out, wounded, but who has learned so much. It has been such a beautiful experience, even though it hurts.

Luke Cline
Founder & CEO, Phielia
Age 19

—●— Phielia